

QA Interview Quick Guide

Top 30 Questions & Answers

Manual Testing · Defects · Agile · Career

FREE TEMPLATE

By Bishal Khatri — QA Engineer

mrbishalkhatri.github.io

How to Use This Guide

■ **Read, don't memorise**

Understand the concept behind each answer. Interviewers probe with follow-up questions — if you've memorised without understanding, you'll get stuck.

■ **Difficulty levels**

Easy (green) = basic knowledge every QA must have. Medium (amber) = shows competence. Hard (red) = demonstrates seniority. All 30 in this guide are achievable with the right preparation.

■ **Use your own words**

Personalise every answer with a real example from your experience. 'In my current project, we...' is 10× more impressive than a textbook answer.

■ **Interview tips**

Each question has an Interview Tip in teal — these contain the nuances that separate good answers from great ones. Read them carefully.

■ **Want all 500 questions?**

This is a free sample of the Complete QA Interview Prep Kit — 500 questions across 10 categories including Automation, API Testing, HRMS, SQL, Mobile, and Behavioural. Get it at mrishalkhatri.github.io/templates.html

Section 1 — Manual Testing Fundamentals

10 Questions
Questions

The core concepts every QA engineer is expected to know cold

Q01 What is software testing and why is it important?

● Easy

Software testing is the process of evaluating an application to verify it meets requirements and to identify defects before users encounter them. It matters because: bugs found in testing are 10–100× cheaper to fix than production bugs, it protects user experience and business reputation, and it gives the team confidence to ship. Testing doesn't prove software is bug-free — it reduces risk to an acceptable level.

■ *Interview tip: Don't just define it — mention the business impact. Interviewers want to see you think beyond 'finding bugs'.*

Q02 What is the difference between Verification and Validation?

● Medium

Verification asks 'Are we building the product right?' — checking that the work matches the specification (e.g., design review, code review, test case review). Validation asks 'Are we building the right product?' — checking that the final product meets the user's actual needs (e.g., UAT, system testing). Verification is static; Validation is dynamic.

■ *Interview tip: Use the V&V; model. This distinction shows ISTQB-level thinking.*

Q03 What are the levels of software testing?

● Easy

Unit Testing — individual functions/modules, done by developers. Integration Testing — interactions between modules, done by QA/dev. System Testing — complete end-to-end system against requirements, done by QA. Acceptance Testing (UAT) — business users validate it meets their needs. Each level catches different defect types and is performed at a different stage of development.

■ *Interview tip: Mention the Testing Pyramid — unit tests should be most numerous, E2E tests fewest.*

Q04 What is the difference between Black Box and White Box testing?

● Easy

Black Box testing tests the software without knowledge of internal code — the tester focuses on inputs and expected outputs (what the system does). White Box testing requires knowledge of the code structure — tests are designed based on code paths, branches, and conditions (how the system does it). Most QA engineers use Black Box; developers use White Box for unit tests. Grey Box combines both.

■ *Interview tip: Mention Grey Box as a bonus — it shows depth.*

Q05 What is the difference between Severity and Priority?

● Medium

Severity measures the technical impact of a bug on system functionality — Critical, High, Medium, Low. Priority measures the business urgency of fixing it. They're independent: a typo on the homepage is Low Severity but High Priority (first thing users see). A critical crash in a rarely-used admin panel is High Severity but Low Priority. Always justify your classifications with business impact.

■ *Interview tip: Use the typo vs crash example — it's the clearest illustration. Almost every interviewer asks this.*

Q06 What makes a good bug report?

● Medium

A good bug report contains: unique Bug ID, clear one-line title, environment details (browser, OS, version), numbered steps to reproduce, expected result, actual result, severity/priority, reproducibility rate, and screenshots or screen recording. The test is: can any developer reproduce the bug in under 5 minutes without asking you anything? If yes — it's a good bug report.

■ *Interview tip: 'Reproducible without asking' is the gold standard. Lead with that.*

Q07 What is Regression Testing and why is it critical?

● Easy

Regression Testing verifies that new code changes haven't broken existing functionality. It's run after every bug fix, new feature, or refactoring. Without regression testing, teams often fix one bug and introduce two new ones without knowing. It's the safety net that allows teams to ship frequently with confidence. Regression is the #1 candidate for automation.

■ *Interview tip: Always link regression testing to automation — it shows strategic thinking.*

Q08 What is the difference between Smoke Testing and Sanity Testing?

● Medium

Smoke Testing (Build Verification Testing) is a broad, shallow test of the most critical features after a new build — it checks if the build is stable enough for further testing. If it fails, the build is rejected. Sanity Testing is narrow and focused — after a specific bug fix, verify that fix works without re-testing everything. Smoke = broad and quick; Sanity = narrow and focused.

■ *Interview tip: Smoke = is this build testable? Sanity = does this specific fix work?*

Q09 What are Boundary Value Analysis and Equivalence Partitioning?

● Medium

Equivalence Partitioning divides input data into groups that should behave identically — test one value from each group. BVA tests at the edges of each partition because bugs cluster at boundaries. For an age input of 18–65: EP gives you test values of 10 (invalid), 35 (valid), 70 (invalid). BVA adds 17, 18, 19 (lower boundary) and 64, 65, 66 (upper boundary). Together they maximise coverage with minimum test cases.

■ *Interview tip: These are the two most important test design techniques — always use them together.*

Q10 What is Exploratory Testing and when do you use it?

● Medium

Exploratory Testing is simultaneous learning, test design, and execution — the tester explores the application using experience and intuition without predefined scripts. It's most effective for: finding bugs that scripted tests miss, testing new features quickly, usability issues, and edge cases. It complements scripted testing — it doesn't replace it. Use time-boxed sessions (e.g., 90-minute charters) to keep it structured.

■ *Interview tip: Frame it as 'structured improvisation' — it shows you know it's disciplined, not random.*

Section 2 — Defects & Test Processes

10 Questions
Questions*Defect lifecycle, test planning, agile QA, and process questions*

Q11 What is the Defect Life Cycle?

● Easy

New → Assigned → Open → In Progress → Fixed → Retest → Closed. Alternative paths: Rejected (not a real bug — works as designed), Duplicate (already reported), Deferred (postponed to next release), Cannot Reproduce (tester cannot replicate). Each state has a responsible party: New (QA), Assigned/Fixed (Dev), Retest/Closed (QA). Track every transition with comments.

■ *Interview tip: Draw this mentally as a state diagram — it shows systematic thinking.*

Q12 What is the difference between a Test Plan and a Test Strategy?

● Medium

Test Strategy is an organisation-level document defining the overall QA approach — what types of testing are done, which tools are used, what standards apply. It's long-lived and applies to all projects. Test Plan is a project-level document detailing scope, schedule, resources, and approach for a specific project or release. Strategy = how we test everything; Plan = how we test this specific project.

■ *Interview tip: Senior candidates know this distinction. Mention you've written both.*

Q13 What are Entry and Exit Criteria?

● Medium

Entry Criteria are preconditions that must be met before testing begins — e.g., test environment ready, build deployed, test cases reviewed and approved, test data prepared. Exit Criteria are conditions that must be met to conclude testing — e.g., pass rate ≥ 95%, no open Critical or High bugs, test summary report signed off. They act as quality gates preventing premature start or stop.

■ *Interview tip: Frame as gates — nothing proceeds without meeting criteria. Shows process maturity.*

Q14 What is Risk-Based Testing?

● Medium

Risk-Based Testing prioritises test effort based on the likelihood and impact of failures. High-risk features (complex, frequently changed, business-critical) get the most testing. Low-risk features get minimal testing. Risk factors: business criticality, code complexity, frequency of change, defect history, regulatory requirements. It's the pragmatic approach when time is limited — and time is always limited.

■ *Interview tip: 'Testing smarter, not more' — that phrase resonates with managers.*

Q15 What is the STLC (Software Testing Life Cycle)?

● Medium

STLC is the sequence of QA activities: 1) Requirement Analysis — review requirements, identify testable conditions, raise queries. 2) Test Planning — scope, approach, resources, schedule. 3) Test Case Design — write and review test cases. 4) Environment Setup — configure test environment, prepare data. 5) Test Execution — run tests, log defects. 6) Test Closure — metrics, lessons learned, sign-off. STLC runs in parallel with SDLC.

■ *Interview tip: Show QA starts at requirements — not after development. This is the shift-left mindset.*

Q16 How do you write effective test cases?

● Easy

Rules: 1) Atomic — test exactly one thing per case. 2) Clear title — format: [Module] Action — Expected Outcome. 3) Explicit preconditions — state what must be true before running. 4) Numbered steps — one user action per step. 5) Specific expected results — no vague outcomes like 'should work'. 6) Include both positive and negative scenarios. The test: two different testers should get the same result running the same case.

■ *Interview tip: If two testers get different results, the test case is broken — not the software.*

Q17 What is a Requirements Traceability Matrix (RTM)?

● Medium

An RTM maps every business requirement to one or more test cases, ensuring complete coverage. It prevents two problems: gold-plating (testing things not required) and coverage gaps (missing requirements). Format: Requirement ID | Requirement Description | Test Case IDs | Test Status. The RTM is QA's proof to stakeholders that all requirements are covered.

■ *Interview tip: RTM is your coverage guarantee — if it's not in the RTM, it wasn't tested.*

Q18 What is Defect Clustering and why does it matter?

● Hard

Defect Clustering (Pareto Principle in testing) states that 80% of defects are found in 20% of the code — usually the most complex or recently changed modules. Smart QA teams identify high-defect-density modules from historical data and focus test effort there. Past defects are a strong predictor of future defects in the same area.

■ *Interview tip: Show you actively analyse defect data to focus testing — it demonstrates data-driven thinking.*

Q19 What is the difference between Functional and Non-Functional testing?

● Easy

Functional testing verifies what the system does — features, business logic, user workflows, input/output behaviour. Non-functional testing verifies how the system performs — speed (performance), security (vulnerability), usability (ease of use), reliability (uptime), scalability (handles growth). Both are equally important. A login that takes 30 seconds is functionally correct but non-functionally broken.

■ *Interview tip: Use the 30-second login example — it makes non-functional testing instantly relatable.*

Q20 How do you decide when to stop testing?

● Hard

Testing is never truly complete — it's a risk management decision. Stop when: exit criteria are met (target pass rate, no critical open bugs), the cost of further testing exceeds the expected benefit, business deadlines are reached, or stakeholders formally accept the risk of remaining defects. Always document what was not tested and explicitly state the risk. Never stop silently.

■ *Interview tip: 'We stop testing when we've managed risk to an acceptable level' — this is the senior answer.*

Section 3 — Agile, Automation & Career

10 Questions

Agile QA role, automation basics, and how to answer tough career questions

Q21 What is Agile and how does QA fit in?

● Easy

Agile is an iterative development approach delivering working software in short sprints (1–4 weeks). QA in Agile is not a separate phase — it's integrated throughout every sprint. QA engineers: review requirements in sprint planning, write test cases during development, test in the same sprint features are built, participate in retrospectives. The goal is a shippable, tested increment at the end of every sprint.

■ *Interview tip: Key phrase: 'Testing happens in the same sprint as development.' This distinguishes agile QA from waterfall QA.*

Q22 What is the role of QA in a Scrum team?

● Easy

In Scrum, QA is part of the cross-functional Development Team — there's no separate QA team or phase. QA responsibilities: participate in sprint planning and backlog refinement, question and clarify acceptance criteria, write and review test cases for each user story, test features as they're built (not at the end), report defects immediately in the same sprint, contribute to Definition of Done, and participate in retrospectives with improvement suggestions.

■ *Interview tip: Emphasise collaborative QA — you're not a gatekeeper but a quality advocate.*

Q23 What is Definition of Done (DoD) in Agile?

● Medium

Definition of Done is a shared checklist that a user story must satisfy before it can be considered 'done' and potentially shippable. A typical DoD includes: code written and reviewed, unit tests passing, functional tests passing, regression tests passing, no open Critical or High bugs, documentation updated, product owner sign-off. QA is central to defining and enforcing DoD — if it's not in the DoD, it won't consistently happen.

■ *Interview tip: Show QA ownership of DoD — it positions you as a quality advocate, not just a tester.*

Q24 What is test automation and what should you automate?

● Medium

Test automation uses scripts to execute tests and verify results automatically. Automate when: tests run repeatedly (regression), tests are data-driven (many combinations), tests are time-consuming manually, tests run across multiple browsers or devices. Don't automate: one-time tests, tests requiring human visual judgment, exploratory sessions, tests for features still changing rapidly. The ROI calculation: (manual effort saved × frequency) > automation build + maintenance cost.

■ *Interview tip: The ROI framing is what managers want to hear.*

Q25 What is the Page Object Model (POM) in Selenium?

● Medium

POM is a design pattern that creates a separate class for each page of the application. Each class contains: the page's element locators and action methods (click, type, verify). Tests call page methods rather than using locators directly. Benefits: changes to UI require updating only one page class (not 50 test files), tests are readable (they describe business actions), and code duplication is eliminated. POM is the industry standard for Selenium frameworks.

■ *Interview tip: If you've used POM, give a real example — it's far more impressive than theory.*

Q26 What is CI/CD and how does testing fit in?

● Medium

CI (Continuous Integration) is the practice of merging code frequently and running automated tests on every commit. CD (Continuous Delivery/Deployment) automatically deploys passing builds to staging or production. Automated tests are the quality gate — the build only proceeds if tests pass. QA owns the test suite and test results; DevOps owns the pipeline infrastructure. Fast, reliable automated tests are what make CI/CD possible.

■ *Interview tip: Position QA as the owner of the CI/CD quality gate — not just a participant.*

Q27 Tell me about a bug you found that had high business impact.

● Easy

Use STAR format: Situation (project context), Task (your role), Action (how you found it — what technique, what testing), Result (what would have happened if it shipped, what actually happened). Example structure: 'During regression testing on our payroll module, I found a calculation error that was adding an extra day's pay to LOP deductions. Across 500 employees it would have overpaid NPR 2.3M. I caught it 2 days before payroll run, filed it as Critical, and worked with dev to fix and retest same day.'

■ *Interview tip: Numbers make this memorable. Prepare your real example before any interview.*

Q28 How do you handle pressure to skip testing and ship faster?

● Hard

Be direct but constructive: 'I understand the deadline pressure. Let me help us make an informed decision.' Then: quantify the risk of skipping (what could break, who it affects, potential cost), propose a risk-based approach (test Critical paths only, defer lower-risk tests), and document the decision formally — if stakeholders choose to ship with known risk, that's their call to make, but it must be explicit and written.

■ *Interview tip: Never say yes to skipping testing without documenting the risk. That protects you and the team.*

Q29 What is your approach to testing a feature with no requirements?

● Hard

First, gather as much context as possible: talk to the developer, product owner, and business stakeholder. Review any existing related features for consistency. Look at competitor products. Then: test based on industry standards and common sense, test all obvious happy paths, test every negative scenario you can think of, focus heavily on exploratory testing, and document your assumptions. Flag the missing requirements formally — undocumented features are a project risk.

■ *Interview tip: Shows initiative and communication skills — two things senior QAs need.*

Q30 Where do you want to be in 5 years as a QA engineer?

● Easy

Be honest and specific. Strong answers: 'I want to grow into a QA Lead role, owning the test strategy and mentoring junior testers while still staying technical.' or 'I want to specialise in automation architecture — building frameworks that serve entire engineering organisations.' or 'I want to move into a SDET role, working closely with developers to build quality into the system from day one.' Connect your answer to the role you're interviewing for.

■ *Interview tip: Show ambition + direction. Vague answers ('just grow') miss the opportunity to impress.*

Want all 500 questions?

This guide covers 30 of the 500 questions in the Complete QA Interview Prep Kit. The full kit covers: Manual Testing, Test Automation, API Testing, HRMS Domain, Agile Processes, Jira Tools, Performance Testing, Mobile Testing, SQL, and Behavioural — everything you need to walk into any QA interview with confidence.

Get it at: mrbishalkhatri.github.io/templates.html

© 2025 Bishal Khatri · Free to use and share · Please credit the source